
Práctica 5: Render pipeline y render features

Objetivo: Practicar con URP

1. Instrucciones

[NO ES IGUAL al de la práctica anterior]

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo escenas con distintos elementos y que utilizan shaders, materiales y renderers distintos.

Se entregará el proyecto completo de Unity, dejando únicamente los directorios necesarios (`Assets`, `Packages` y `ProjectSettings`). En el directorio `Assets` habrá, además, una carpeta `Capturas` con imágenes de pantalla que muestren el resultado de cada una de las tareas pedidas.

2. Punto de partida

[NO ES IGUAL al de la práctica anterior]

Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline*. Añade un directorio `URPSettings` para sus assets. Antes de empezar con la primera parte:

- Crea un asset “*URP Asset (with Universal Renderer)*” en la carpeta anterior llamado `PR05_URenderPipelineAsset`. Creará también un *renderer* que se utilizará por defecto `PR05_URenderPipelineAsset_Renderer`.
- Configura el proyecto (opción *Graphics* y quizá en las *Quality Settings*) para que utilice esa configuración.

En la primera parte de la práctica la cámara de las distintas escenas utilizarán ese renderer (es preferible *forzar su uso* en lugar de dejar el renderer por defecto, que cambiaremos para hacer pruebas cómodamente). Posteriormente cada escena tendrá un renderer distinto, para lo que habrá que:

- Crear el nuevo renderer “*URP Universal Renderer*” con el nombre concreto indicado en la carpeta `URPSettings`.



Figura 1: Contenido del directorio Assets de la entrega

- Añadir el renderer en la lista de renderers de `PR05_URRenderPipelineAsset`.
- Seleccionar ese renderer en la cámara de la escena.
- Aunque no es obligatorio, seguramente quieras poner el nuevo renderer como el renderer por defecto para que se utilice también en la pestaña *Scene* (si no has forzado el uso del renderer inicial en las primeras escenas, esta acción alterará su funcionamiento).

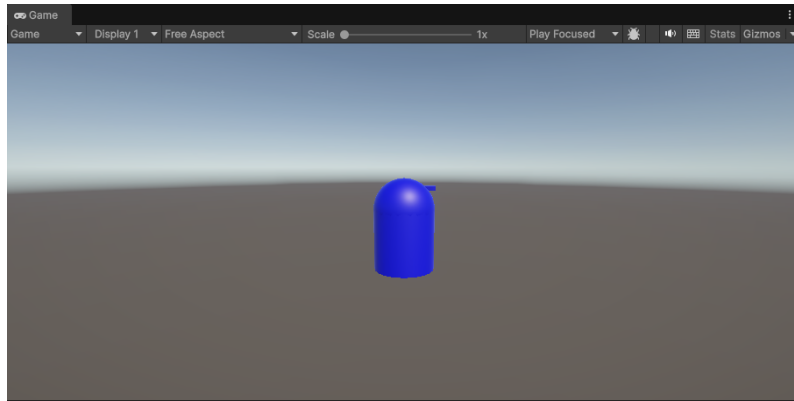
Crea en la carpeta *Materials* los siguientes materiales, basados en el shader por defecto de URP, Universal Render Pipeline/Lit:

- Azul que utilice el RGB (0, 0, 255).
- Naranja con (255, 120, 0).
- GrisClaro con (200, 200, 200).

Crea un objeto para el “jugador” (puedes hacerlo prefab, si quieres), que estará compuesto de un cilindro, una esfera y un cubo (todos ellos Azules). Los tres serán objetos pertenecientes a un `GameObject` vacío, colocado en (0, 0, 0). Las propiedades de cada uno son:

- Cilindro / cuerpo: L: (0, 0.3, 0), S: (0.6, 0.3, 0.6).
- Esfera / cabeza: L: (0, 0.6, 0), S: (0.6, 0.6, 0.6).
- Cubo / nariz: L: (0.2, 0.8, 0), S: (0.2, 0.05, 0.05).

Por último, coloca la cámara en L: (0, 1, -3) y R: (10, 0, 0).



3. Pintando el frame en un cubo

Cuando URP termina de dibujar la geometría opaca y pintar el skybox, hace una copia del resultado a una textura auxiliar llamada `_CameraOpaqueTexture` a la que puede accederse desde los shaders. Para familiarizarte con el *Frame Debugger*, puedes buscar la pasada en la que se copia. Si no la encuentras, puede que en la configuración del Render Pipeline Asset no esté activada la opción “Opaque texture”.

Crea un nuevo shader (y material) llamado `01FrameEnCubo` similar al shader que pinta una textura de prácticas anteriores pero que utilice esa textura auxiliar. El shader *no* necesita tener como parámetro/propiedad la textura, pues Unity la hace disponible a todos los shaders. Para utilizarla:

- Incluye el fichero `DeclareOpaqueTexture.hlsl`, disponible en el mismo directorio que `Core.hlsl`
- Utiliza la función `SampleSceneColor(uv)` para leer de ella.

Crea un cubo en L: (2, 0.5, 0) que utilice el material. Si no te funciona, utiliza el *Frame debugger* para buscar dónde puede estar el error.

4. Habitación misteriosa

Para la siguiente práctica necesitaremos una escena con una geometría distinta. En concreto una “habitación” abierta por los cuatro lados, con dos objetos dentro. El objetivo es que dependiendo del lado desde el que se mire al interior, se verá uno u otro objeto.

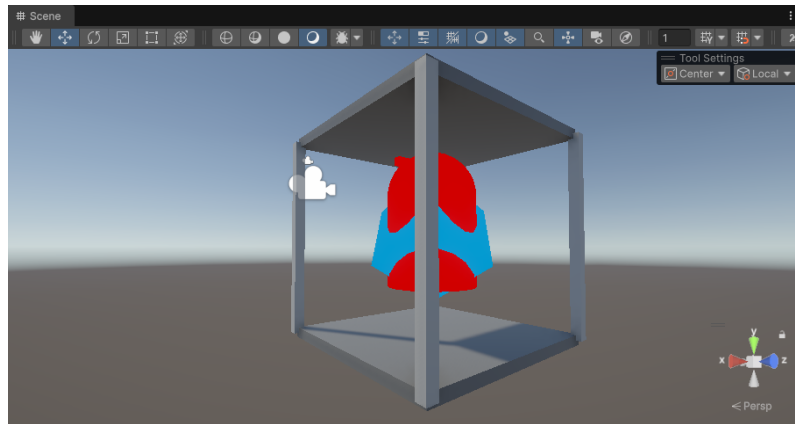
Para la escena, crea un objeto vacío en L: (0, 0, 0) donde se añaden los siguientes cubos, todos con el material `GrisClaro`:

- Cuatro columnas, con escalas S: (0.1, 2, 0.1) y posiciones L: (-1, 0, -1), L: (-1, 0, 1), L: (1, 0, 1) y L: (1, 0, -1).
- El suelo y techo, con S: (2, 0.1, 2) y L: (0, 1, 0) y L: (0, -1, 0).

Coloca la cámara en L: (0, 1, -3) y R: (25, 0, 0).

Para los objetos interiores, vamos a poner al jugador en la posición L: (0, -0.5, 0) con escala S: (1.5, 1.5, 1.5) y un cubo en L: (0, 0, 0), con rotación R: (45, 45, 45) y escala S: (0.8, 0.8, 0.8). En lugar de utilizar los materiales que ya tenemos preparados, recupera el shader `01ColorFijo` de la primera práctica (renómbralo a `02ColorFijo`). Recordarás que el

shader permitía especificar en el material el color fijo que se utilizaría. Crea dos materiales, uno rojo (02ColorFijo_Rojo RGB 255, 0, 0) y otro azul claro (02ColorFijo_Azul RGB 0, 160, 255) y usalos para el jugador y esfera respectivamente.



El efecto que queremos conseguir se hace gracias al *stencil buffer*:

- En cada lado de la habitación pondremos planos invisibles ocupando todo el lateral que, en lugar de pintarse en el *color buffer*, únicamente escribe en el stencil. Los valores escritos son distintos dos a dos, de forma que dos lados enfrentados pueden escribir un 1 y los otros dos un 2.
- Se extienden los materiales anteriores para que hagan una comprobación de stencil de forma que sólo se pinte la geometría si el valor del stencil coincide con uno dado.

Los planos que puedes poner (en la jerarquía de la habitación) tienen escala S: (0.2, 1, 0.2) y con posiciones y rotaciones:

- L: (0, 0, -1), R: (-90, 0, 0).
- L: (0, 0, 1), R: (90, 0, 0).
- L: (-1, 0, 0), R: (0, 0, 90).
- L: (1, 0, 0), R: (90, 0, -90).

El shader que escribe en el stencil buffer (pero no en el color buffer) es el siguiente (02StencilMask):

```
Shader "Unlit/02StencilMask"
{
    Properties
    {
        [IntRange] _StencilRef("Stencil Ref", Range(0, 255)) = 1
    }
    SubShader
    {
        Tags {
            "RenderType" = "Opaque"
            "Queue" = "Geometry-1"
            "RenderPipeline" = "UniversalPipeline"
        }
    }
}
```

```

    Pass
    {
        Tags { "LightMode" = "UniversalForward" }

        Stencil {
            Ref[_StencilRef]
            Comp Always
            Pass Replace
        }

        ZWrite Off
    }
}

```

Fíjate que:

- Recibe como parámetro (`_StencilRef`) el valor que escribirá en el stencil buffer.
- Fuerza que los objetos que lo utilicen se “pinten” antes que el resto de la geometría opaca (cola `Geomtery-1`).
- La comparación con el stencil siempre tiene éxito (`Always`) y siempre reemplaza el valor que hubiera por el del parámetro (`Ref[_StencilRef]`).

Crea dos materiales, `02StencilMask_1` y `02StencilMask_2` que lo utilicen y que escriban el valor 1 y 2 en el stencil respectivamente. Utiliza el primero en dos lados enfrentados y el segundo en los otros dos.

El último paso es extender el shader `02ColorFijo` añadiendo un parámetro entero `_StencilRef` igual que el del shader de la máscara y añadir en la configuración del Stencil *de la pasada* que solo se pintará la geometría si los valores son iguales a los del parámetro y que el valor en el stencil no se modificará:

```

Stencil {
    Ref[_StencilRef]
    Comp Equal
    Pass Keep
    Fail Keep
}

```

La captura que incluyas debe dejar claro que el efecto está funcionando.

5. Jugador siempre visible

En la práctica 3 hicimos un efecto que hacía que el jugador apareciera siempre visible: cuando quedaba por detrás de algún objeto, aparecía la geometría en un color fijo.

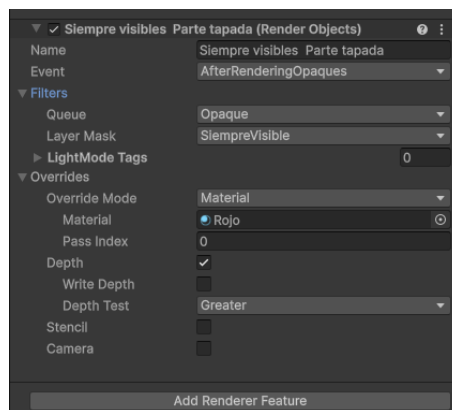
Para hacerlo nos aprovechamos de la posibilidad de dar dos materiales para renderizar el objeto. Sin embargo requería que en el shader del color fijo se configurara el test de z-buffer distinto. Eso limita poder utilizar cualquier material preexistente para mostrar al jugador tapado.

Una alternativa mejor, en URP, es utilizar la *Render Objects feature*.

Crea una escena similar a la del `01FrameEnCubo` pero sustituyendo el cubo por un plano `GrisClaro` con R: (-90, 0, 0) y S: (0.1, 0.1, 0.1). Crea un Universal Renderer `03JugadorSiempreVisible` y asegurate de que lo utiliza la cámara de la escena.

Los pasos para conseguir el efecto son:

- Crea un material Rojo basado en el shader *Universal Render Pipeline/Unlit* de color rojo (al ser *Unlit* no le afectará la luz).
- Añade una nueva *Layer* en la *User Layer 3* llamada *SiempreVisible* y asígnasela al jugador (y recursivamente a toda su geometría).
- Añade una Render Feature para dibujar, *AfterRenderingOpakes*, los objetos que *SiempreVisible* forzando que el material a utilizar no sea el del objeto sino el Rojo. Hay que sobrescribir también la opción de profundidad para que en lugar de utilizar la del shader, se utilice como test el *Greater* y no se escriba en el z-buffer.



¿Ha salido el efecto que esperabas? Utiliza el *Frame debugger* para entender porqué pasa lo que pasa y arréglalo.

Haz una captura del *Frame Debugger* en el que se vean las distintas pasadas y el resultado final de la escena.

6. Habitación imposible - Solución general

El efecto de la habitación imposible adolecía del mismo problema que el jugador siempre visible de la práctica 3: los materiales de los objetos imposibles debían usar shaders que permitieran configurar el stencil.

Gracias a las *Render features* de URP se puede conseguir el mismo efecto con cualquier material, sobrescribiendo el uso del Stencil en la fase de dibujado.

Para hacerlo:

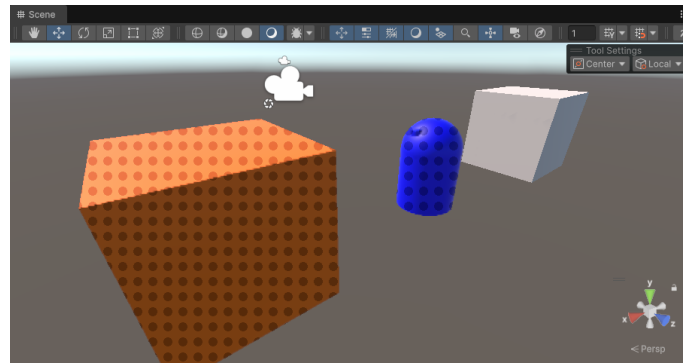
- Clona la escena de la habitación a otra `04HabitacionMagicaGeneral`.
- Restaura el material del jugador para que utilice el azul basado en el shader general de URP.
- Crea un nuevo renderer `04HabitacionMagicaGeneral` que sea el que utilice la cámara.
- Cambia el material del cubo interior para que sea `GrisClaro`.
- Añade dos capas nuevas, `Imposible1` e `Imposible2` y asignaselas a jugador y cubo.

- Añade las *Render features* en el renderer necesarias para conseguir el efecto.

Igual que en la práctica anterior, incluye en la entrega una captura en la que se vea el *Frame debugger* con las distintas pasadas y el resultado final del frame.

7. Lunares

El objetivo de esta práctica es hacer que todos los objetos que pertenezcan a la *layer* EfectoEspecial sean dibujados con unos “lunares”: círculos situados a modo de trama en la pantalla que oscurecen la zona ocupada por el objeto (multiplican su color por 0.5):



El efecto se basa en un shader 05Lunares con los parámetros:

```
Properties
{
    [IntRange] _Dist("Distancia entre círculos", Range(0, 100)) = 20
    [IntRange] _Radio("Radio", Range(0, 50)) = 6
}
```

El shader lee el color de la textura con la información del frame tras el dibujado de opacos (igual que el primero, 01FrameEnCubo) y lo oscurece dependiendo del valor de los parámetros (dados en píxeles).

Recuerda que la forma de convertir una posición en *screen space* a píxeles es:

```
float2 screenUV = i.posSS / i.posSS.w;
float2 screenXY = screenUV * _ScreenParams.xy;
```

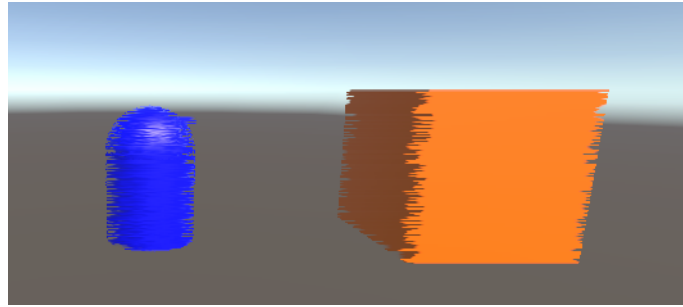
Tendrás que:

- Crear una nueva escena 05Lunares similar a la primera con un cubo adicional en la posición L: (-2, 0.5, 0). Uno de ellos puede ser GrisClaro y el otro Naranja.
- Crear la *User layer* 8 llamada EfectoEspecial y asignársela a algunos de los objetos.
- Crear un nuevo renderer 05Lunares y añadir la *Render feature* necesaria.

8. Glitch

En el último efecto debes añadir una perturbación en los objetos que pertenezcan a la etiqueta EfectoEspecial anterior:

La idea de funcionamiento es similar a la del efecto de los lunares pero ahora el shader tendrá un único parámetro:



```
Properties
{
    [IntRange] _MaxDesplX("Maximo glitch horizontal (pixeles)", Range(0, 100)) = 10
}
```

Y utilizando la función de ruido ya utilizada con anterioridad y la parte entera del tiempo (`_Time.w - trunc(_Time.w)`), elegirá un desplazamiento horizontal para cada Y de la pantalla de forma que desplazará esa fila un número de píxeles entre 0 y `_MaxDesplX`.

9. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- El proyecto de Unity con los directorios mínimos necesarios (`Assets`, `Packages` y `ProjectSettings`). El contenido del directorio `Assets` debe ser el que se deduce de las instrucciones anteriores y que puede verse en la figura 1.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.